

1. What is Inheritance?

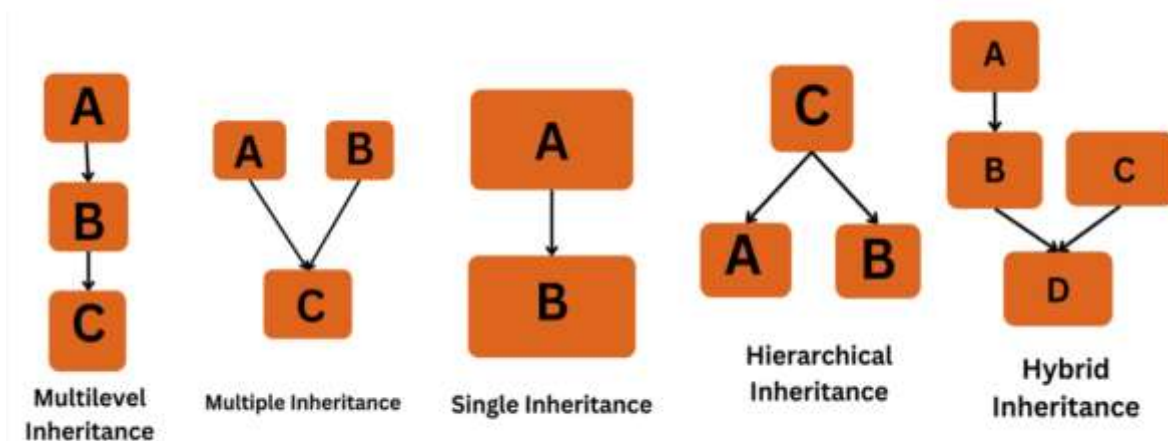
- Inheritance is an OOP feature that allows one class to acquire the attributes and methods of another class.
- In simple words: A child class can reuse the code of an existing parent class.

2. Why Do We Use Inheritance?

- The most important reason is: Code Reusability
- **Other reasons**
 - o Code reusability
 - o Avoid code duplication
 - o Easier maintenance
 - o Create relationships between classes
 - o Extend existing functionality

3. Types of Inheritance

- 1) Single Inheritance
- 2) Multilevel Inheritance
- 3) Multiple Inheritance
- 4) Hierarchical Inheritance
- 5) Hybrid Inheritance



Types of Inheritance in Python

4. Basic Inheritance

```
class Parent:
```

```
    pass
```

```
class Child(Parent):
```

```
    pass
```

```
class Animal:

    def eat(self):
        print("Animal eats")

class Dog(Animal):

    def bark(self):
        print("Dog barks")
```

5. Parent and Child Both Have Constructors

```
class Parent:

    def __init__(self):
        print("Parent Constructor")

class Child(Parent):

    def __init__(self):
        print("Child Constructor")

c = Child()
```

o/p - Child Constructor

- Because the child has its own `__init__()`.

- The child's constructor overrides/hides the parent's constructor for normal initialization of the child object.

6. Only Parent Has Constructor

```
class Parent:
    def __init__(self):
        print("Parent Constructor")

class Child(Parent):
    pass

c = Child()
```

o/p - Parent Constructor

- Because Child does not have its own `__init__()`.

7. What is super()?

- `super()` is used to access functionality from a parent class inside a child class.

It is commonly used to call:

- Parent constructor
- Parent methods

```
class Person:
    def __init__(self, name):
        self.name = name

class Student(Person):
    def __init__(self, name, marks):
        super().__init__(name)
        self.marks = marks
```

8. super() with Normal Method

```
class Animal:
    def sound(self):
        print("Animal makes sound")

class Dog(Animal):
    def sound(self):
        super().sound()
        print("Dog barks")

d = Dog()

d.sound()
```

9. Method Overriding

- Method overriding occurs when a child class provides its own implementation of a method that already exists in the parent class.

```
class Animal:
    def sound(self):
        print("Animal makes sound")

class Dog(Animal):
    def sound(self):
        print("Dog barks")

d = Dog()
d.sound()
```

10. Access Modifiers in Python

- Python doesn't have Java-style strict public, protected, and private keywords.
- Instead, naming conventions are used.

1. Public

2. Protected

3. Private

1) Public

- A public member can be accessed from inside and outside the class.

```
class Student:

    def __init__(self):
        self.name = "Rahul"

s = Student()

print(s.name)
```

2) Protected

- Written with a single _.

```
class Student:

    def __init__(self):
        self._marks = 80
```

3) Private

- Written with two leading underscores:

```
class Student:

    def __init__(self):
        self.__marks = 80

print(s.__marks)
```

11. Getter and Setter

1) Getter

- A getter is a method used to read/access the value of an attribute.

```
def get_marks(self):  
    return self.__marks
```

2) Setter

- A setter is a method used to update/change the value of an attribute.

```
def set_marks(self, marks):  
    if 0 <= marks <= 100:  
        self.__marks = marks
```

12. Types of Inheritance

- 6) Single Inheritance
- 7) Multilevel Inheritance
- 8) Multiple Inheritance
- 9) Hierarchical Inheritance
- 10) Hybrid Inheritance

1) Single Inheritance

- One parent → One child.

```
class Animal:  
  
    def eat(self):  
        print("Eating")  
  
class Dog(Animal):  
  
    def bark(self):  
        print("Barking")
```

```
d = Dog()

d.eat()
d.bark()
```

2) Multilevel Inheritance

- Inheritance happens in multiple levels.

Grandparent

↓

Parent

↓

Child

```
class Grandparent:

    def house(self):
        print("House")

class Parent(Grandparent):

    def car(self):
        print("Car")

class Child(Parent):

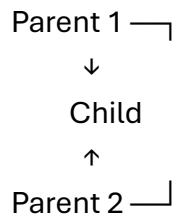
    def bike(self):
        print("Bike")

c = Child()

c.house()
c.car()
c.bike()
```

3) Multiple Inheritance

- One child → Multiple parents.



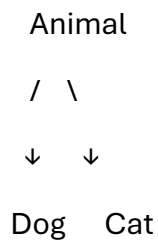
```
class Father:
    def father_property(self):
        print("Father property")

class Mother:
    def mother_property(self):
        print("Mother property")

class Child(Father, Mother):
    pass
```

4) Hierarchical Inheritance

- One parent → Multiple children.



```
class Animal:
    def eat(self):
        print("Eating")

class Dog(Animal):
    def bark(self):
```



```
        print("Barking")

class Cat(Animal):

    def meow(self):
        print("Meowing")
```

5) Hybrid Inheritance

- Hybrid inheritance is a combination of two or more types of inheritance.



```
class A:
    pass

class B(A):
    pass

class C(A):
    pass

class D(B, C):
    pass
```